

# Open-RMF 패키지 역할 정리 — rmf\_demos vs libi

ABA(도서관 책 배달 매니플레이터) Open-RMF 학습 repo · 2026-06-23

## 1. rmf\_demos 패키지 (기본 레퍼런스 7개)

| 패키지                     | 빌드타입         | 역할 (상세)                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------------------|--------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| rmf_demos               | ament_cmake  | 데모 전체를 묶는 <b>launch</b> 진입점. <code>common.launch.xml</code> 하나가 RMF 코어를 통째로 켜다 — <code>rmf_traffic_schedule</code> (일정DB)· <code>rmf_traffic_blockade</code> (좁은구간중재)· <code>building_map_server</code> (/map)· <code>rmf_task_dispatcher</code> (경매)· <code>visualization</code> (rviz)· <code>door/lift supervisor</code> . 그 위에 <code>office.launch</code> · <code>hotel.launch</code> 등 <b>시나리오별 launch</b> + <code>tinyRobot_config.yaml</code> 같은 fleet config 보관. |
| rmf_demos_assets        | ament_cmake  | <b>Gazebo 3D 모델·미디어</b> . <code>TinyRobot</code> · <code>DeliveryRobot</code> 모델 SDF(slotcar 플러그인 장착)·메시, <code>dispenser/ingestor</code> (물건 주고받는 워크셀), 환경 오브젝트. RMF 데모가 gz에 띄우는 <b>시각/물리 자산 전부</b> .                                                                                                                                                                                                                                                                   |
| rmf_demos_bridges       | ament_python | <b>RMF ↔ 외부 통신 스택 브리지</b> . <code>fleet_socketio_bridge</code> (웹 대시보드 ↔ RMF, socket.io), <code>fleet_robotmanager_mqtt_bridge</code> (MQTT 로봇매니저 ↔ RMF). RMF 토픽을 <b>웹/IoT 프로토콜로 중계</b> . ※ <b>domain_bridge</b> 아님!                                                                                                                                                                                                                                                     |
| rmf_demos_fleet_adapter | ament_python | <b>EasyFullControl 어댑터(RMF↔로봇 통역)</b> . <code>fleet_adapter.py</code> (RMF 등록· <code>navigate/execute_action</code> 콜백)· <code>fleet_manager.py</code> (slotcar 백엔드, <code>PathRequest</code> 발행/ <code>robot_state</code> 수신)· <code>RobotClientAPI.py</code> (HTTP 다리). <code>config+navgraph</code> 읽어 <code>add_easy_fleet</code> .                                                                                                                                  |
| rmf_demos_gz            | ament_cmake  | <b>Gazebo(gz) 시뮬 launch</b> . gz world 띄우고 로봇 spawn, <code>ros_gz</code> bridge 연결. 코어(rmf_demos)와 <code>sim(gz)</code> 을 결합하는 시나리오 <code>launch</code> ( <code>office.launch.xml</code> 등).                                                                                                                                                                                                                                                                             |
| rmf_demos_maps          | ament_cmake  | <b>데모 맵</b> . <code>traffic_editor</code> <code>building.yaml</code> + floorplan 이미지. 빌드 시 <code>building_map_generator</code> 로 <b>world + navgraph(0.yaml)</b> 산출.                                                                                                                                                                                                                                                                                                     |
| rmf_demos_tasks         | ament_python | <b>task 요청 CLI</b> . <code>dispatch_patrol</code> · <code>loop</code> · <code>delivery</code> · <code>clean</code> · <code>action</code> · <code>go_to_place</code> + <code>cancel</code> . task JSON을 조립해 <code>ApiRequest</code> 로 <code>task_api_requests</code> 에 발행(submitter).                                                                                                                                                                                     |

## 2. 우리 패키지 (libi\_rmf\_\* · 5개)

| 패키지                    | 빌드타입         | 역할 (상세)                                                                                                                                                                                                                                                                                                    | rmf_demos 대응               |
|------------------------|--------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|
| libi_rmf_maps          | ament_cmake  | 우리 맵 <code>new_map</code> (library). <code>building.yaml</code> → <code>navgraph</code> + <b>M1 좌표 보정</b> (pgm윌드↔ <code>traffic_editor</code> <code>navgraph</code> 대응점 뉘음변환). <code>charger-point_a/b</code> waypoint 명명.                                                                                 | rmf_demos_maps ✓           |
| libi_rmf_fleet_adapter | ament_python | <b>EasyFullControl 직역 포팅</b> . 4파일 그대로 + 패키지명/config만 libi로. slotcar 백엔드(M2~4) → M5에 nav2로 백엔드만 교체. <code>execute_action</code> 에 <b>pickup/dropoff</b> 팔 동작(M4) 추가.                                                                                                                                       | rmf_demos_fleet_adapter ✓  |
| libi_rmf_tasks         | ament_python | <code>dispatch_delivery.py</code> — <b>배달 compose task 조립·제출</b> ( <code>go(shelf)</code> → <code>pickup</code> → <code>go(desk)</code> → <code>dropoff</code> ). 우리만의 task 정의가 사는 곳.                                                                                                                      | rmf_demos_tasks ✓          |
| libi_rmf_bringup       | ament_cmake  | <b>RMF 두뇌 조립 launch</b> . <code>libi_rmf.launch.xml</code> = 코어( <code>schedule</code> · <code>blockade</code> · <code>building_map_server</code> · <code>dispatcher</code> ) + 어댑터 <code>include</code> . <code>scripts/rmf.sh</code> 가 띄움. M5엔 <code>nav2</code> · <code>domain_bridge</code> 까지 여기 합쳐짐. | rmf_demos(common.launch) ✓ |
| libi_rmf_bridge        | ament_cmake  | <b>domain_bridge 설정(M5)</b> . 실로봇이 각자 ROS 도메인에서 <code>nav2</code> 폴스택을 돌릴 때, RMF에 필요한 토픽만 선택적으로 도메인 간 브리지. <b>M5에서만</b> 등장.                                                                                                                                                                                | rmf_demos_bridges 아님 ⚠     |

⚠ 혼한 오해 — `libi_rmf_bridge` ≠ `rmf_demos_bridges` :

- `rmf_demos_bridges` = RMF ↔ **웹/MQTT**(socketio·mqtt) 외부 통신 브리지
- `libi_rmf_bridge` = **domain\_bridge**(실로봇 각자 ROS 도메인 ↔ RMF, M5) — upstream의 별도 도구. rmf\_demos가 안 가진 거라 우리가 새로 둔 것. 둘 다 "브리지"지만 목적이 완전히 다름.

### 3. 빠진 패키지 + 왜 합리적인 선택인가

| 빠진 rmf_demos 패키지                                   | 그 역할은 어디서 채우나                                                                                           | 왜 빠는 게 합리적                                                                                                                                       |
|----------------------------------------------------|---------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>rmf_demos_assets</b><br>(로봇/물체 3D 모델)           | controller/pinky_pro/pinky_description<br>(pinky URDF) + slotcar 플러그인은 <b>apt 설치</b><br>(libslotcar.so) | 우리 로봇은 데모 TinyRobot이 아니라 <b>pinky(실 타깃)</b> . 로봇 자산은 <b>로봇과 함께 controller/</b> 에 둔다(pingdergarten: 장비=controller, 관제=fleet). 데모용 자산을 따로 둘 이유 없음. |
| <b>rmf_demos_gz</b><br>(Gazebo 시물 launch)          | controller/pinky_pro/pinky_gz_sim +<br>scripts/libi_sim.sh (tmux 래퍼, MODE 토글)                           | 시물은 <b>로봇에 종속</b> → pinky_gz_sim(로봇 옆)에. RMF 관제 (fleet/)와 분리해 책임이 명확. world는 pgm_to_gazebo(M1)로 생성.                                              |
| <b>rmf_demos_bridges</b><br>(MQTT/socketio 브리지)    | service/ · app/ · web/ ( <b>비-ROS, 현재 placeholder 미구현</b> )                                             | 이건 <b>외부 웹/IoT 연동</b> 계층 → ABA에선 <b>비-ROS 앱</b> 이 담당(ROS 바깥). 학습 초점은 RMF 코어라 <b>나중/별도</b> . (domain_bridge와 혼동 주의 ⚠)                             |
| ( <b>rmf_demos</b> 모놀리식 자체)<br>launch+config 한 덩어리 | libi_rmf_bringup (조립 launch) + config는<br>libi_rmf_fleet_adapter/config                                 | 데모 시나리오 다발(office/hotel...) 대신 우리는 <b>단일 맵</b> → bringup 하나로 충분. 패키지 책임을 쪼개 더 명확하게.                                                              |

### 4. 그대로 인용(포팅)한 것 vs 우리가 직접 구축한 것

open-rmf(rmf\_demos) 코드를 어디까지 가져왔고, 어디부터 우리 고유 작업인지.

| 구분      | 항목                                                    | 출처 · 우리가 바꾼/만든 부분                                                                                                                                  |
|---------|-------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|
| 인용 · 포팅 | <b>fleet_adapter / fleet_manager / RobotClientAPI</b> | rmf_demos_fleet_adapter <b>4파일</b> 직역 포팅(거의 그대로). 바꾼 것: 패키지명·config 경로·상대 import, 그리고 execute_action 에 <b>pickup/dropoff 케이스</b> 만 추가.             |
| 인용 · 포팅 | <b>libi_fleet_config.yaml</b> (구조)                    | rmf_demos_tinyRobot_config.yaml <b>틀 그대로</b> . 값은 우리 것: pinky 속도/footprint, library charger-waypoint, reversible:false, actions:[pickup,dropoff] . |
| 인용 · 포팅 | <b>dispatch_delivery.py</b> (골격)                      | rmf_demos_tasks_dispatch_action.py 의 <b>ApiRequest 발행·compose</b> 조립 패턴 차용. <b>배달 시퀀스(go→pickup→go→dropoff)</b> 조립은 우리 것.                          |
| 인용 · 포팅 | <b>slotcar 플러그인 파라미터</b>                              | TinyRobot slotcar 기본값 차용. 바꾼 것: tire_radius · base_width (pinky 실측), reversible:false (후진금지), 질량.                                                  |
| 직접 구축   | <b>M1 좌표보정 도구</b>                                     | fix_navgraph.py · make_navgraph.sh · show_navgraph.py — pgm월드↔navgraph 닳음 변환(대응점). <b>rmf_demos에 없는 우리 고유</b> (RMS≈1cm).                           |
| 직접 구축   | <b>library 맵 일습</b>                                   | new_map.{pgm,png,yaml,building.yaml,pairs.yaml} — 우리가 그린 도서관 맵.                                                                                    |
| 직접 구축   | <b>실행 스크립트</b>                                        | libi_sim.sh (tmux sim+rviz, MODE 토글) · rmf.sh (bringup 래퍼) · libi_kill.sh (정리).                                                                    |
| 직접 구축   | <b>libi_rmf_bringup launch</b>                        | libi_rmf.launch.xml — 코어+어댑터 조립을 우리가 작성.                                                                                                           |
| 직접 구축   | <b>pinky-slotcar 통합</b>                               | pinky_slotcar.urdf.xacro · pinky_gz_slotcar.urdf.xacro — pinky 몸체에 slotcar 붙이는 변형 + diffdrive/slotcar 토글.                                          |
| 직접 구축   | <b>M3 2번째 로봇 spawn</b>                                | launch_sim · libi_sim.sh 에 조건부 2대 spawn (negotiation 관찰).                                                                                          |
| 직접 구축   | <b>RViz TF 연결 fix</b>                                 | sim_view.launch.py 정적변환(map→world, base_link→base_footprint) — gz↔rviz 좌표 불일치 해결.                                                                  |
| 직접 구축   | <b>M4 팔 핸들러</b>                                       | perform_arm_action (mock 로그+딜레이 후 finished) — 우리가 추가한 동작 핸들러.                                                                                      |
| 직접 구축   | <b>문서</b>                                             | 트러블슈팅.md · README · docs/ · 이 PDF — 학습 기록.                                                                                                         |

**요약: 인용 · 포팅** = RMF↔로봇 핵심 배선(어댑터 3종) + 설정/요청 골격 — "검증된 표준을 그대로". **직접 구축** = 맵·좌표보정·로봇통합·실행환경·팔동작·문서 — "우리 로봇(pinky)·맵·task에 맞춘 부분". → **바퀴(표준)는 빌려오고, 우리 시나리오에 맞춘 살은 직접 붙였다.**

#### 핵심 한 줄

맵·어댑터·tasks·bringup은 rmf\_demos를 거의 1:1 미러링, 빠진 3개(assets·gz·bridges)는 **역할이 만 데 있어서 뻔** 것:

- 로봇 자산·시물(assets/gz) → 로봇과 함께 **controller/pinky\_pro** (우리 로봇은 pinky라 이미 있음)
- 외부 웹/MQTT 브리지(bridges) → 비-ROS **service/app/web** (앱 계층, 미구현)

- **libi\_rmf\_bridge** 는 새로 추가 (M5 domain\_bridge) — rmf\_demos\_bridges와는 다른 브리지
- "RMF 관제(fleet/)에 필요한 것만 미러링 + 로봇·시뮬은 **controller/**, 외부연동은 **app/**으로 분리" — pingdergarten 컨벤션을 따른 합리적 선택.